

第四章讲义：栈和队列

栈和队列都是逻辑上的数据结构，所谓逻辑的，就是与物理无关的，再说得简单一点，无论链表还是数组都可以实现栈和队列。

1. 栈

栈 (Stack) 的特点是**后进先出**。

比如一堆餐盘，放的时候总是要放在最上面，拿的时候，也是先拿最上面的。



如果将每个餐盘看成一个“元素”，就得到了栈。

1.1 用链表实现栈

```
1  typedef struct{
2      int data;
3      struct stack *next;
4  }stack;
5
6
7  /*只能放到最末尾*/
8  stack* push(stack *head, stack *element){
9      if(head == 0){
10         head = element;
11         return head;
12     }
```

```

13     stack *p = head;
14     stack *temp = head;
15     while( p != 0){
16         temp = p;
17         p = p->next;
18     }
19     temp->next = element;
20     return head;
21 }
22 /**
23 pop不是简单的删掉，而是还需要得到响应的值，
24 由于head指针是必须返回的，所以就有了两个返回值
25 因此，这里面head必须要用二维指针。
26 如果大家能想明白就想，想不明白也没关系，不影响后面的学习。
27 但是可能会影响408考研。
28 */
29 stack *pop(stack **head){
30     if(*head == 0){
31         return 0;
32     }
33     stack *p = *head;
34     stack *temp = *head;
35     while( p->next != 0){
36         temp = p;
37         p = p->next;
38     }
39     temp->next = 0;
40     if(p == *head){
41         *head == 0;
42     }
43     return p;
44 }

```

1.2 用数组实现栈

用数组实现栈就简单多了，我们只要定义一个top位置就可以了。

```

1  #include <iostream>
2  #define MAX 100
3  using namespace std;
4  int push(int stack[],int top,int data){
5      if(top + 1 == MAX){
6          cerr<<"stack is full";
7          return -1;
8      }
9      stack[top] = data;
10     return top+1;
11 }
12

```

```

13 int pop(int stack[],int *top){
14     if(*top == 0){
15         cerr<<"stack is empty";
16         return -1;
17     }
18     int data = stack[*top - 1];
19     *top = *top - 1;
20     return data;
21 }
22
23
24 int main(){
25     int stack[100] = {0}; // 定义一个足够大的数组
26     int top = 0;
27
28     // 入栈
29     top = push(stack,top,100);
30     top = push(stack,top,123);
31
32     cout<<stack[0]<<endl<<stack[1]<<endl;
33
34     // 出栈
35     //pop(stack,&top);
36     cout<<pop(stack,&top)<<endl;
37     cout<<pop(stack,&top)<<endl;
38
39     return 0;
40 }

```

1.3 用vector实现栈

```

1  #include <iostream>
2  #define MAX 100
3  #include <vector>
4  using namespace std;
5  int main(){
6
7
8      vector<int> s;
9      //放的时候，永远要放在最后面
10     s.push_back(1);
11     s.push_back(2);
12
13     /*取的时候，一定要记得删掉最后一个元素*/
14     int data = s.back();
15     s.pop_back();
16     cout<<data<<endl;

```

```

17
18     cout<<s.back();
19     s.pop_back();
20
21
22     return 0;
23 }

```

1.4 STL本身就提供了栈

```

1  #include <iostream>
2  #include <stack>
3
4  int main() {
5      std::stack<int> s;
6
7      // 向栈中添加元素
8      s.push(10);
9      s.push(20);
10     s.push(30);
11     std::cout<<s.top()<<std::
12         endl;
13
14     // 打印栈顶元素
15     std::cout << "Top element is: " << s.top() << std::endl; // 输出: Top element is: 30
16
17     // 移除栈顶元素
18     s.pop();
19     std::cout << "After popping, top element is: " << s.top() <<
20     std::endl; // 输出: After popping, top element is: 20
21
22     // 检查栈是否为空
23     if (!s.empty()) {
24         std::cout << "Stack is not empty." << std::endl; // 输出:
25         Stack is not empty.
26     }
27
28     // 打印栈的大小
29     std::cout << "Size of stack: " << s.size() << std::endl; // 输出: Size of stack: 2
30
31     // 继续移除元素
32     s.pop();
33     s.pop();
34
35     // 检查栈是否为空
36     if (s.empty()) {

```

```

35     std::cout << "Stack is empty." << std::endl; // 输出:
    Stack is empty.
36     }
37
38     return 0;
39 }

```

1.5 栈的使用

栈有什么用，我们用STL中stack来实现一个案例：**判断一个字符串中的括号是否正确**

```

1  #include <iostream>
2  #define MAX 100
3  #include <stack>
4  #include <string>
5  using namespace std;
6  bool isBracketRight(string str){
7      if(str.size() == 0){
8          return true;
9      }
10     stack<char> bracket;
11     int i = 0 ;
12     for(;i<str.size();i++){
13         if(str[i] == '('){
14             bracket.push(str[i]);
15             continue;
16         }
17         if(str[i] == ')'){
18             if(bracket.top() == '(') {
19                 bracket.pop();
20             }
21         }
22     }
23     if(bracket.empty()){
24         return true;
25     }
26     return false;
27 }
28
29 int main(){
30
31     string s = "1+2((2)))";
32     cout<<isBracketRight(s)<<endl;
33     return 0;
34 }

```